Project 1

Voting System (using HashMap)

```
import java.util.*;
import java.io.*;


class Candidate {
   private String name;

   public Candidate(String name) {
      this.name = name;
   }

   public String getName() {
      return name;
   }
}


class VotingSystem {
   private HashMap<String, Integer> voteMap;
   private HashSet<String> voters; // To prevent duplicate voting

   public VotingSystem() {
      voteMap = new HashMap<>();
      voters = new HashSet<>();
   }


   public void addCandidate(String name) {
      voteMap.put(name, 0);
      System.out.println("Candidate added: " + name);
   }


   public void castVote(String voterId, String candidateName) {
      try {
         if (voters.contains(voterId)) {
            throw new Exception("You have already voted!");
         }

         if (!voteMap.containsKey(candidateName)) {
            throw new Exception("Candidate not found!");
         }
```

```
        voteMap.put(candidateName, voteMap.get(candidateName) + 1);
        voters.add(voterId);

        System.out.println("Vote cast successfully!");

    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

public void displayResults() {
    System.out.println("\n--- Voting Results ---");
    for (Map.Entry<String, Integer> entry : voteMap.entrySet()) {
        System.out.println(entry.getKey() + " : " + entry.getValue());
    }
}


public void announceWinner() {
    String winner = null;
    int maxVotes = -1;

    for (Map.Entry<String, Integer> entry : voteMap.entrySet()) {
        if (entry.getValue() > maxVotes) {
            maxVotes = entry.getValue();
            winner = entry.getKey();
        }
    }

    if (winner != null) {
        System.out.println("\nWinner is: " + winner + " with " + maxVotes + " votes!");
    } else {
        System.out.println("No votes cast yet.");
    }
}


public void saveResultsToFile() {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter("results.txt"))) {
        for (Map.Entry<String, Integer> entry : voteMap.entrySet()) {
            writer.write(entry.getKey() + " : " + entry.getValue());
            writer.newLine();
        }
        System.out.println("Results saved to file successfully!");
    } catch (IOException e) {
        System.out.println("File Error: " + e.getMessage());
    }
```

```
    }
}


public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        VotingSystem vs = new VotingSystem();

        while (true) {
            System.out.println("\n1. Add Candidate");
            System.out.println("2. Cast Vote");
            System.out.println("3. Display Results");
            System.out.println("4. Announce Winner");
            System.out.println("5. Save Results");
            System.out.println("6. Exit");

            System.out.print("Enter choice: ");
            int choice = sc.nextInt();
            sc.nextLine(); // consume newline

            switch (choice) {
                case 1:
                    System.out.print("Enter candidate name: ");
                    String name = sc.nextLine();
                    vs.addCandidate(name);
                    break;

                case 2:
                    System.out.print("Enter voter ID: ");
                    String voterId = sc.nextLine();
                    System.out.print("Enter candidate name: ");
                    String candidate = sc.nextLine();
                    vs.castVote(voterId, candidate);
                    break;

                case 3:
                    vs.displayResults();
                    break;

                case 4:
                    vs.announceWinner();
                    break;

                case 5:
                    vs.saveResultsToFile();
                    break;
```

```
            case 6:
                System.out.println("Exiting...");
                return;

            default:
                System.out.println("Invalid choice!");
        }
    }
  }
}
```

project 2

Student Record Management System

```
import java.util.*;

import java.io.*;

import java.time.LocalDate;


// OOP: Student Class

class Student {

  String id, name, email;


  public Student(String id, String name, String email) {

    this.id = id;

    this.name = name;

    this.email = email;

  }


  // Converts object to a single line for the text file

  public String toFileString() {

    return id + "," + name + "," + email;

  }
```

```
}

public class Main {
    static List<Student> studentList = new ArrayList<>();
    static Map<String, List<String>> attendanceMap = new HashMap<>();
    static Scanner input = new Scanner(System.in);

    // File Names
    static final String STUDENT_FILE = "students.txt";
    static final String ATTENDANCE_FILE = "attendance.txt";

    public static void main(String[] args) {
        loadData(); // Load text files at start

        int choice;
        do {
            System.out.println("\n--- MANUAL MANAGEMENT SYSTEM ---");
            System.out.println("1. Add Student    2. Delete Student");
            System.out.println("3. Mark Presence  4. View Report");
            System.out.println("5. Display All    6. Save & Exit");
            System.out.print("Enter Choice: ");
            choice = Integer.parseInt(input.nextLine());

            switch (choice) {
                case 1 -> addStudent();
                case 2 -> deleteStudent();
                case 3 -> markAttendance();
                case 4 -> viewReport();
                case 5 -> studentList.forEach(s -> System.out.println(s.id + " | " + s.name));
                case 6 -> saveData();
            }
```

```
    } while (choice != 6);
}

// --- MANUAL METHODS ---

static void addStudent() {
    System.out.print("ID: "); String id = input.nextLine();
    System.out.print("Name: "); String name = input.nextLine();
    System.out.print("Email: "); String email = input.nextLine();
    studentList.add(new Student(id, name, email));
    System.out.println("Student Added Locally.");
}

static void deleteStudent() {
    System.out.print("Enter ID to remove: ");
    String id = input.nextLine();
    studentList.removeIf(s -> s.id.equals(id));
    attendanceMap.remove(id);
    System.out.println("Removed.");
}

static void markAttendance() {
    System.out.print("Enter Student ID: ");
    String id = input.nextLine();
    String today = LocalDate.now().toString();

    attendanceMap.putIfAbsent(id, new ArrayList<>());
    if (!attendanceMap.get(id).contains(today)) {
        attendanceMap.get(id).add(today);
        System.out.println("Attendance marked for " + today);
    } else {
```

```
        System.out.println("Already marked for today.");
    }
}

static void viewReport() {
    System.out.print("Enter Student ID: ");
    String id = input.nextLine();
    List<String> dates = attendanceMap.getOrDefault(id, new ArrayList<>());
    System.out.println("Total Days Present: " + dates.size());
    System.out.println("Dates: " + dates);
}

// --- MANUAL FILE HANDLING (TEXT BASED) ---

static void saveData() {
    try {
        // Save Students
        PrintWriter studentWriter = new PrintWriter(new FileWriter(STUDENT_FILE));
        for (Student s : studentList) {
            studentWriter.println(s.toFileString());
        }
        studentWriter.close();

        // Save Attendance
        PrintWriter attWriter = new PrintWriter(new FileWriter(ATTENDANCE_FILE));
        for (String id : attendanceMap.keySet()) {
            attWriter.println(id + ":" + String.join(",", attendanceMap.get(id)));
        }
        attWriter.close();

        System.out.println("All data saved to .txt files.");
```

```
    } catch (IOException e) {
        System.out.println("Save Error: " + e.getMessage());
    }
}

static void loadData() {
    try {
        // Load Students
        File sf = new File(STUDENT_FILE);
        if (sf.exists()) {
            Scanner reader = new Scanner(sf);
            while (reader.hasNextLine()) {
                String[] data = reader.nextLine().split(",");
                studentList.add(new Student(data[0], data[1], data[2]));
            }
            reader.close();
        }

        // Load Attendance
        File af = new File(ATTENDANCE_FILE);
        if (af.exists()) {
            Scanner reader = new Scanner(af);
            while (reader.hasNextLine()) {
                String[] parts = reader.nextLine().split(":");
                String id = parts[0];
                String[] dates = parts[1].split(",");
                attendanceMap.put(id, new ArrayList<>(Arrays.asList(dates)));
            }
            reader.close();
        }
    } catch (Exception e) {
```

```
            System.out.println("Load Error: " + e.getMessage());
        }
    }
}
```

Project 3

Attendance Tracking System

```
import java.io.*;
import java.text.SimpleDateFormat;
import java.util.*;

public class AttendanceSystem {

    // Map: Student -> List of Dates Present
    private static HashMap<String, List<String>> attendance = new HashMap<>();

    private static final String FILE_NAME = "attendance.txt";
    private static final SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");

    public static void main(String[] args) {
        loadFromFile();

        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n--- Attendance System ---");
            System.out.println("1. Mark Attendance");
```

```
        System.out.println("2. View Report");
        System.out.println("3. Calculate Percentage");
        System.out.println("4. Monthly Summary");
        System.out.println("5. Exit");
        System.out.print("Enter choice: ");
        choice = sc.nextInt();
        sc.nextLine();

        switch (choice) {
            case 1:
                markAttendance(sc);
                break;
            case 2:
                viewReport();
                break;
            case 3:
                calculatePercentage(sc);
                break;
            case 4:
                monthlySummary(sc);
                break;
            case 5:
                saveToFile();
                System.out.println("Exiting...");
                break;
            default:
                System.out.println("Invalid choice!");
        }

    } while (choice != 5);
}
```

```
// 1. Mark Attendance
private static void markAttendance(Scanner sc) {
    String date = sdf.format(new Date());

    System.out.print("Enter number of students: ");
    int n = sc.nextInt();
    sc.nextLine();

    for (int i = 0; i < n; i++) {
        System.out.print("Enter student name: ");
        String name = sc.nextLine();

        System.out.print("Present (y/n): ");
        String present = sc.nextLine();

        if (present.equalsIgnoreCase("y")) {
            attendance.putIfAbsent(name, new ArrayList<>());
            attendance.get(name).add(date);
        }
    }

    saveToFile();
    System.out.println("Attendance marked for " + date);
}

// 2. View Report
private static void viewReport() {
    System.out.println("\n--- Attendance Report ---");

    for (String student : attendance.keySet()) {
```

```
        System.out.println(student + " -> " + attendance.get(student));
    }
}

// 3. Calculate Attendance Percentage
private static void calculatePercentage(Scanner sc) {
    System.out.print("Enter student name: ");
    String name = sc.nextLine();

    if (!attendance.containsKey(name)) {
        System.out.println("No record found.");
        return;
    }

    List<String> presentDays = attendance.get(name);

    // Count total unique days in system
    Set<String> allDates = new HashSet<>();
    for (List<String> dates : attendance.values()) {
        allDates.addAll(dates);
    }

    double percentage = (presentDays.size() * 100.0) / allDates.size();

    System.out.println("Attendance Percentage of " + name + " = " + percentage + "%");
}

// 4. Monthly Summary
private static void monthlySummary(Scanner sc) {
    System.out.print("Enter month (MM): ");
    String month = sc.nextLine();
```

```
    System.out.println("\n--- Monthly Summary ---");

    for (String student : attendance.keySet()) {
        long count = attendance.get(student).stream()
                .filter(date -> date.substring(5, 7).equals(month))
                .count();

        System.out.println(student + " -> " + count + " days present");
    }
}

// Save to file
private static void saveToFile() {
    try (PrintWriter pw = new PrintWriter(new FileWriter(FILE_NAME))) {
        for (String student : attendance.keySet()) {
            pw.print(student + ":");
            List<String> dates = attendance.get(student);
            pw.println(String.join(",", dates));
        }
    } catch (IOException e) {
        System.out.println("Error saving file.");
    }
}

// Load from file
private static void loadFromFile() {
    File file = new File(FILE_NAME);
    if (!file.exists()) return;

    try (BufferedReader br = new BufferedReader(new FileReader(file))) {
```

```
            String line;

            while ((line = br.readLine()) != null) {
                String[] parts = line.split(":");
                String name = parts[0];
                List<String> dates = new ArrayList<>();

                if (parts.length > 1 && !parts[1].isEmpty()) {
                    dates = new ArrayList<>(Arrays.asList(parts[1].split(",")));
                }

                attendance.put(name, dates);
            }
        } catch (IOException e) {
            System.out.println("Error loading file.");
        }
    }
}
```